Electricity Consumption Forecasting for Tetouan, Morocco

Shuyuan Ye (251089284), Ruojing Chen (25141869), Taorong Tang (251495944), Yunzhao Li (251139575)
DATASCI 9000 – Introduction to Machine Learning
Western University, London, Canada

***Abstract*—This paper examines short-term electricity consumption forecasting for Tetouan, Morocco, using a suite of classical and modern machine learning models. Starting from a cleaned and feature-engineered version of the public Tetouan load dataset, we compared Multiple Linear Regression, Ridge Regression, fully-connected Multi-Layer Perceptron (MLP), three convolutional neural network (CNN) variants, and XGBoost. We evaluated these models in a consistent preprocessing pipeline and a temporal split 70/15/15 train–validation-test while using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 as evaluation metrics. Our analysis demonstrates that simple linear models provide interpretable baselines but struggle to capture complex temporal dynamics; sequence-based CNNs significantly improve over these baselines, while XGBoost achieves the strongest overall performance with RMSE test of approximately 685 and $R^2 \approx 0.998$ on total consumption.**

***Index Terms*—load forecasting, time series, convolutional neural networks, XGBoost, regression, Tetouan**

I. Introduction

Short-term forecasting of electric load is crucial for the operation of the power system, demand management, and smart-grid planning. Accurate predictions help utilities schedule generation, reduce operating costs, and mitigate blackouts. At the city scale, load is driven by both temporal patterns (hour of day, day of week, season) and exogenous factors such as temperature and solar radiation.

In the project, we focused on the Tetouan city power consumption dataset from northern Morocco, which provides high-frequency measurements of electricity usage in three distribution zones together with meteorological variables and timestamps. We aimed to build and compare several supervised learning models that forecast total consumption in the next time step.

Compared with our earlier progress report, the final report (i) consolidates the full preprocessing pipeline, (ii) describes the architecture and training procedure of each model we implemented in Python, and (iii) presents a comparative analysis of empirical performance, which is supported by tables and plots that can be directly exported from our code and inserted into the indicated placeholders.

II. Related Work

Traditional approaches to load forecasting include linear regression and autoregressive integrated moving average (ARIMA) models, which assume relatively simple temporal dynamics. With the increasing availability of smart-meter data, the machine learning methods such as Random Forests, Gradient Boosting, and Support Vector Regression have become crucial since they can model non-linear relationships between predictors and demand.

Deep learning models, especially recurrent neural networks (RNNs) and convolutional neural networks (CNNs), have recently been applied to the electricity consumption data. CNN-based approaches are able to learn local temporal patterns by applying filters over sliding windows of past observations; gradient-boosted trees, such as XGBoost, achieve strong performance with relatively modest feature engineering.

Our work follows this line of research by directly comparing interpretable linear models, feedforward neural networks, CNN architectures, and XGBoost on the same Tetouan dataset and evaluation protocol.

III. Data and Preprocessing

A. Dataset

We used the Tetouan city power consumption dataset from the UCI Machine Learning Repository, which contains measurements recorded at regular intervals over the year 2017. Each record includes:

- Three zone-level electricity consumptions (`Zone1`, `Zone2`, `Zone3`);
- Meteorological variables such as temperature, humidity, and wind speed;
- Solar radiation and atmospheric pressure;
- A timestamp providing date and time.

Following our progress report, we define the *total consumption* target variable as the sum of the three zone consumptions:

$$y_t = \texttt{Zone1}_t + \texttt{Zone2}_t + \texttt{Zone3}_t.$$

B. Cleaning and Feature Engineering

The main preprocessing steps are:

- Correct column names and ensuring numeric types for all continuous variables;
- Construct a unified `DateTime` variable from the date and time columns and sorting records chronologically;
- Remove rows with missing values in any required predictor or the target;
- Engineer time-based features (hour of day, day of week, and month) from the timestamp;
- Standardize continuous predictors using z-score normalization based on the training set statistics.

For models that require fixed-length sequences (CNNs and some neural networks), we transformed the time series into overlapping windows of length L (e.g. $L = 24$ time steps):

$$\mathbf{x}_t = [(\mathbf{f}_{t-L+1}), \ldots, (\mathbf{f}_t)],$$

where $\mathbf{f}_t$ denotes the feature vector at time t. The corresponding label y_t is the total consumption at the final time step of the window.

C. Train–Validation–Test Split

We respected the temporal ordering of the data to avoid leakage. The dataset is split chronologically into:

- 70% for training, used to fit model parameters;
- 15% for validation, used for hyperparameter selection and early stopping;
- 15% as a held-out test set for final evaluation.

All models share the above split to ensure a fair comparison.

IV. Experimental Setup

We evaluated all models using the same standardized features and targets. For each model, we record the Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2) on the test set:

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \tag{1}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}, \tag{2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2}. \tag{3}$$

Hyperparameters for each model are selected using the validation set; in practice this is implemented through manual search and, for some models, light grid search over small sets of candidate values (e.g., regularization strengths, learning rates, tree depths).

V. Models and Per-Model Analysis

In this section, we analyzed each model implemented in Python. For each model, we briefly explained the architecture, how it uses the engineered features, and its empirical behavior on the Tetouan data.

A. Multiple Linear and Ridge Regression

The Multiple Linear Regression (MLR) model serves as the simplest baseline. It assumes a linear relationship between the non-sequential feature vector $\mathbf{f}_t$ at time t and the target y_t:

$$\hat{y}_t = \beta_0 + \boldsymbol{\beta}^\top \mathbf{f}_t.$$

Ridge Regression extends MLR by adding an L2 penalty on the coefficients:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_t (y_t - \beta_0 - \boldsymbol{\beta}^\top \mathbf{f}_t)^2 + \lambda\|\boldsymbol{\beta}\|_2^2,$$

where $\lambda \geq 0$ controls the strength of regularization. In our implementation, we do not regularize the bias term and select λ based on validation performance.

Quantitative behavior: On the validation set, the linear baseline achieves MAE $\approx$ 8269.8, RMSE $\approx$ 10487.5, and $R^2 \approx 0.393$, with Ridge giving almost identical numbers, indicating that moderate regularization reduces coefficient magnitude but does not drastically change predictive performance. In the test set, the (OLS) regression baseline achieves MAE = 7085.38, RMSE = 8807.08, and $R^2 = 0.618$. These scores reveal that linear models capture a substantial portion of the variance (over 60% on tests) but leave considerable room for improvement.

The fitted coefficients (for example, OLS and Ridge intercepts around 73,426 and similar slopes) suggest sensible relationships between time/weather features and load, confirming that these models are useful for interpretation even if not the most accurate predictors.

B. Multi-Layer Perceptron (MLP)

The MLP is a fully-connected feedforward neural network that maps the feature vector $\mathbf{f}_t$ to the target. A typical architecture used in our code is:

- Input layer of dimension equal to the number of features;
- One or more hidden layers with non-linear activations (e.g., ReLU);
- An output layer with a single neuron for the scalar prediction.

We trained the MLP using backpropagation with an optimizer such as Adam and mean squared error (MSE) loss. Regularization is achieved through weight decay and/or dropout, and early stopping based on validation loss.

Quantitative behavior: Aggregating over 5 independent runs, the MLP reaches an average test MAE of 8885.81 ± 772.33, RMSE of 11028.16 ± 1078.32, and R^2 of 0.396 ± 0.125. Surprisingly, this is *worse* than the linear baseline both in terms of absolute error and variance explained. The relatively large standard deviations indicate some instability across runs.

Hence, it suggests that our current MLP configuration may be over-parameterized or insufficiently regularized for the available data. That simple fully connected networks without explicit sequence modeling are not the most suitable choice for this task.

C. CNN Architectures (v1, v2, v3)

To better exploit temporal structure in the data, we implemented several one-dimensional CNN models that operate on sliding windows of past feature vectors.

1) CNN v1: CNN v1 is a simple architecture with:

- One or two 1D convolutional layers with small kernel size (e.g., 3–5) applied along the time dimension;
- A pooling layer to reduce temporal resolution;
- One or two fully-connected layers mapping the flattened feature maps to the output.

Over multiple runs, CNN v1 achieves average test MAE of 8201.06 ± 201.30, RMSE of 10102.23 ± 126.22, and R^2 of 0.498 ± 0.013. This already outperforms the MLP and increases R^2 by approximately 10 percentage points compared with the linear baseline, indicating that explicit sequence modeling is beneficial.

2) CNN v2: CNN v2 deepens the architecture and increases the number of filters per layer. It may also incorporate:

- Additional convolutional blocks to capture longer-range dependencies;
- Batch normalization to stabilize training;
- Dropout for regularization.

CNN v2 performs substantially better than CNN v1, achieving average test MAE of 6838.39 ± 203.72, RMSE of 8477.56 ± 232.93, and R^2 of 0.646 ± 0.020. Compared to the linear baseline, this reduces RMSE by about 3.7% and boosts R^2 from 0.618 to around 0.65, while also decreasing MAE by approximately 3.6%.

3) CNN v3: CNN v3 is our most advanced convolutional model. It includes:

- Multiple convolutional layers with varying kernel sizes to capture multi-scale patterns;
- Residual or skip connections between blocks;
- Carefully tuned filter counts and learning rate based on validation performance.

CNN v3 reaches average test MAE of 6943.35 ± 343.52, RMSE of 8536.85 ± 440.21, and R^2 of 0.641 ± 0.037. Its performance is similar to CNN v2's: slightly better RMSE but worse MAE on average, and overlapping R^2 confidence intervals. The higher standard deviations suggest that CNN v3 might be more sensitive to initialization and training conditions.

Overall, CNN v2 and v3 clearly outperform both the linear baseline and the MLP, confirming that convolutional architectures are effective for this load forecasting task.

D. XGBoost

XGBoost is an efficient implementation of gradient-boosted decision trees. It builds an ensemble of regression trees in a stage-wise manner, where each new tree attempts to correct the residuals of the previous trees. ITs key hyperparameters include maximum tree depth, number of trees, learning rate, and subsampling ratios.

We feeded XGBoost the same feature vectors as the MLR and MLP models and evaluated both on a single zone and on the total consumption.

Quantitative behavior: In Zone 1 alone, XGBoost achieves RMSE test of 397.886, MAE of 288.160, and R^2 of 0.9957; while in total power consumption (`Zone1 + Zone2 + Zone3`), XGBoost achieves RMSE of 685.337, MAE of 492.957, and R^2 of 0.9976 on the test set. These values are an order of magnitude better than those of the CNNs and linear models in terms of absolute error, and R^2 is very close to 1, indicating that XGBoost captures nearly all of the explainable variance.

This result highlights the strength of tree-based ensembles on tabular data with mixed temporal and meteorological inputs. XGBoost appears to be the most suitable model when predictive accuracy is the main objective.

VI. Results and Model Comparison

Table I summarizes the test-set performance of all models on total consumption using MAE, RMSE, and R^2. CNN metrics and MLP metrics are averaged over multiple runs and reported with standard deviations.

A. Linear vs. Regularized Linear Models

Comparing OLS and Ridge shows that, with our current feature set, adding an L2 penalty does not drastically change performance: both models reach RMSE test around 8800 and $R^2 \approx 0.62$. This indicates that the feature space is not severely ill-conditioned and that overfitting is somewhat limited at the linear level. However, the residual errors remain large, suggesting that linear structure alone is insufficient to model the complex dependence of load on time and weather.

B. Linear vs. Non-linear Models

The MLP was expected to improve over the linear baseline by capturing non-linear interactions. Instead, its average performance is worse (higher MAE and RMSE, lower R^2), and highly variable across the runs. This hightlights optimization and regularization challenges: the network may be too deep or wide for the data, leading to overfitting or convergence to suboptimal local minima.

By contrast, non-linear tree ensembles (XGBoost) dramatically outperform linear models, with RMSE dropping from roughly 8800 to 685 and R^2 increasing from about 0.62 to 0.998. This confirms that non-linear structure is crucial for this task and that gradient-boosted trees are well-suited well suited to capturing it in tabular data.

C. Sequence-aware CNNs vs. Point-wise Models

The CNN models (v1–v3) explicitly consume sequences of past observations and therefore can internalize temporal patterns such as daily cycles, weekday/weekend differences, and short-term persistence. Compared to the point-wise MLP, all CNN variants improve R^2 and reduce MAE/RMSE, demonstrating the value of sequence modeling.

Among CNNs, v2 and v3 clearly outperform v1, with v2 slightly ahead in MAE and v3 slightly ahead in RMSE. Their R^2 values around 0.64–0.65 are also higher than the linear baseline (0.618), albeit still far from XGBoost's 0.9976. This suggests that sequence-aware deep learning models are competitive but, in our current implementation, not yet fully optimized for this dataset.

D. Overall Ranking and Trade-offs

Across all models, we can summarize the trade-offs as follows:

- **Predictive accuracy:** XGBoost is the clear winner, followed by CNN v2/v3, then the linear baseline, CNN v1, and finally the MLP.

TABLE I
TEST-SET PERFORMANCE ON TOTAL CONSUMPTION. CNN AND MLP RESULTS ARE AVERAGES OVER MULTIPLE RUNS ($\pm$ STANDARD DEVIATION).

Model	MAE	RMSE	R^2
Linear Regression (OLS baseline)	7085.38	8807.08	0.618
Ridge Regression (baseline)	7085.38	8807.08	0.618 (approx.)
MLP (5 runs)	8885.81 ± 772.33	11028.16 ± 1078.32	0.396 ± 0.125
CNN v1	8201.06 ± 201.30	10102.23 ± 126.22	0.498 ± 0.013
CNN v2	6838.39 ± 203.72	8477.56 ± 232.93	0.646 ± 0.020
CNN v3	6943.35 ± 343.52	8536.85 ± 440.21	0.641 ± 0.037
XGBoost (total consumption)	492.957	685.337	0.9976

- **Interpretability:** Linear models are easiest to interpret via coefficients; XGBoost offers moderate interpretability through feature importance; CNNs and MLPs are more opaque.
- **Computational cost:** Linear and Ridge Regression train almost instantly. XGBoost and the MLP require moderate training time, while CNN v2/v3 are the most computationally demanding due to their sequence-based convolutions.

Depending on the application, people might choose XGBoost for the highest accuracy, a CNN for sequence-aware modeling where tree-based methods are less convenient, or a linear model when interpretability is prioritized over raw performance.

VII. DISCUSSION

Our experiments illustrate several practical lessons for load forecasting:

- Simple linear models provide strong baselines and useful interpretability, particularly when data is limited or model transparency is essential; however, they leave substantial residual variance.
- Regularization (Ridge) is crucial but not yet sufficient on its own to close the performance gap with more expressive models.
- Non-linear models such as CNNs and XGBoost can capture complex interactions between weather and time, yielding significantly better accuracy; among them, tree-based ensembles currently dominate in our setting.
- Sequence-based CNNs are capable of leveraging the temporal structure of the data and offer a flexible framework for extending to multi-step or multi-output forecasting.

Future work could explore more advanced architectures (e.g., attention-based models or hybrid CNN–RNN structures), more systematic hyperparameter optimization, and probabilistic forecasts that quantify uncertainty.

VIII. CONCLUSION

We presented a comparative study of multiple models for short-term electricity consumption forecasting in Tetouan, Morocco. Building on a consistent preprocessing pipeline and a temporal 70/15/15 split, we analyzed linear regression, Ridge regression, an MLP, several CNN architectures, and XGBoost.

Our results show that while linear models form an essential baseline and offer interpretability, modern non-linear methods—especially XGBoost and sequence-based CNNs – better exploit meteorological and temporal information to improve forecast accuracy.

ACKNOWLEDGMENT

We appreciate the course instructor and teaching assistants for guidance, and the creators of the Tetouan dataset for making the data publicly available.

REFERENCES